Document Number:	P0912R2
Date:	2018-06-08
Audience:	WG21
Revises:	P0912R1
Reply to:	gorn@microsoft.com

Merge Coroutines TS into C++20 working draft

Abstract

This paper proposes merging Working Draft of Coroutines TS [N4736] into the C++20 working draft [N4741].

Revision history

r0: initial revision

r1:

- Markdown rendered as HTML.
- Replaced ligature ff with two letters ff.
- Expanded on compiler availability.
- Reworded the closing paragraph.
- Make insertions green.
- Updated working draft numbers

r2:

- ... keywords s/were/are added ...
- P0664R3 => P0664R4
- Added list of edits requested by LWG on 2018-06-08.

Introduction

- Coroutines of this kind were available in a shipping compiler since 2014.
- We have been receiving and acting upon developer feedback for 4 years.
- The software built using coroutines has been deployed on more than **400 million** devices in the hands of delighted consumers.
- The software built using coroutines on Linux and Windows is powering the foundational services of Windows Azure cloud services.
- Coroutines have been used by thousands of software developers.

- We have two publicly available implementations of coroutines TS in MSVC since version 2015 SP2 (2016) and clang version 5 (2017).
- We have an independent coroutine implementation in EDG frontend (2015).
- There are several open source coroutine abstraction libraries, including an extensive open source coroutine library cppcoro that utilizes most of the coroutine features with tests that can be used to smoke test emerging coroutine implementation for completeness and correction.
- Other libraries provides coroutines TS bindings for its types, for example, Facebook's Folly, Just::Thread Pro and others.

Coroutines address the dire need by dramatically simplifying development of asynchronous code. Coroutines have been available and in use for 4 years. We have implementations from two major compiler vendors. It is time to merge Coroutines TS to the working paper.

Wording

- Apply coroutine wording from N4736 to the working draft with the following changes:
    - replace `experimental::` with nothing
    - replace `<experimental/coroutine>` with `<coroutine>`
    - in synopsis in [coroutine.trivial.awaitables] and [support.coroutine]
        - remove "namespace experimental {" and "} // namespace experimental"
        - remove "inline namespace coroutines_v1 {" and "} // namespace coroutines_v1"
- Apply changes from "LWG requested edits on 2018-June-08" section below.
- Apply resolution of issue 29 from Coroutine TS issue list P0664R4.
- Add underlined text to rationale in [diff.cpp17.lex]:

> **Rationale:** Required for new features. The `requires` keyword is added to introduce constraints through a *requires-clause* or a *requires-expression*. The `concept` keyword is added to enable the definition of concepts (17.6.8). The `co_await`, `co_yield`, and `co_return` keywords are added to enable the definition of coroutines (11.4.4). Effect on original feature: Valid ISO C++ 2017 code using `concept`, `co_await`, `co_yield`, `co_return`, or `requires` as an identifier is not valid in this International Standard.

- Add underlined text to "Effect on original feature" in [diff.cpp17.library]:

> **Effect on original feature:** The following C++ headers are new: `<compare>`, `<coroutine>`, and `<syncstream>`. Valid C++ 2017 code that #includes headers with these names may be invalid in this International Standard.

LWG requested edits on 2018-June-08

- Modify paragraph 1 of 21.11.1.1/[coroutine.traits.primary] as follows:

> The header <experimental/coroutine> defines the primary template coroutine_traits such that if ArgTypes is a parameter pack of types and ~~if R is a type that has a valid (17.8.2) member type promise_type~~, if the *qualified-id* R::promise_type is valid and denotes a type (17.9.2), then coroutine_traits<R,ArgTypes...> has the following publicly accessible member:

- Modify paragraph 2 of 21.11.1.1/[coroutine.traits.primary] as follows:

> ~~All~~Program defined specializations of this template shall define a publicly accessible nested type named promise_type.

- In clause 21.11 in titles of the subclauses replace "Struct template" with "Class template"
- In clause 21.11 template introducers replace "typename" with "class"
- Modify the paragraph 3 in subclause 21.11.2.3/[coroutine.handle.observers] as follows:

> *Returns:* ~~true if~~ address() != nullptr ~~, otherwise false~~ .

- In clause 21.11 capitalize the first letter of the text in all elements (Requires, Effects, etc) unless its an expression.
- Make synchronization element in paragraphs 3 and 6 of 21.11.2.4/[coroutine.handle.resumption] a note as follows:

> ~~Synchronization: a~~ *[Note: A concurrent resumption of the coroutine via resume, operator(), or destroy may result in a data race. --endnote]*

- Modify paragraph 2 of 21.11.2.6/[coroutine.handle.compare] as follows:

> *Returns:* less<~~void*~~>()(x.address(), y.address()).

- Reorder relational operators in synopsis 21.11/[support.coroutine] and in subclause 21.11.2.6/[coroutine.handle.compare] to be in the following order: ==, !=, <, >, <=, >=.
- Modify 21.11.2.11/[coroutine.handle.noop.address] as follows:

> *Remarks:* A noop_coroutine_handle's ptr is always ~~contains~~ a non-null pointer value.

- Add a space in 21.11.3/[coroutine.promise.noop] as follows:

> struct noop_coroutine_promise_{};